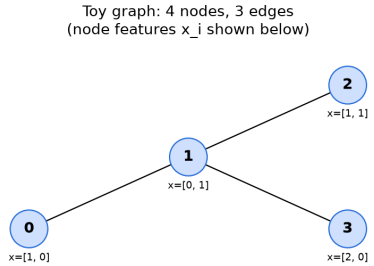
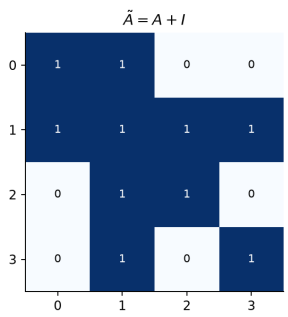
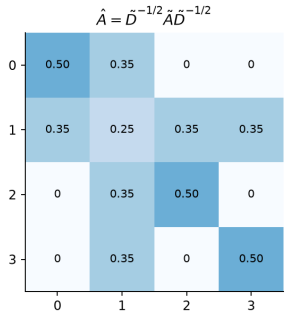
Three-Layer Vanilla Graph Convolutional Network

Derivation, Implementation and Evaluation for Node Classification on the Cora Citation Network







Assignment phase	Where in this report
Phase 1 – Basic understanding of the Vanilla GCN	Sections 1, 3, 4, 5 (graph basics, aggregation, ingredients, architecture, diagrams)
Phase 2 – Mathematical concepts and algorithm notes	Section 6 (derivation, three-layer equations, loss, back-propagation, pseudocode, worked example)
Phase 3 – Implementation	Sections 7, 8, 9 and the GitHub repository (NumPy + PyTorch code, outputs, results)

Integrity statement. Every number in this report was produced by running the accompanying code and is stored in the `outputs/` folder of the repository. The trained results come from the NumPy implementation, which was executed. A PyTorch implementation of the same model is included, but it could **not** be run in the environment where this package was prepared, so no PyTorch result is claimed. Fixed-weight demonstrations (Section 6.9) are never presented as trained results.

1. Introduction and Problem Statement

Many datasets are networks rather than tables or images: papers cite papers, users follow users, proteins interact with proteins. A node in such a network is described by its own attributes, but a second, equally informative source of evidence is *who it is connected to*. Ordinary classifiers see only the first source. A **Graph Convolutional Network (GCN)** combines both by repeatedly averaging each node's representation with those of its neighbours.

This project studies the vanilla GCN of Kipf and Welling with **exactly three graph-convolution layers**. The practical task is **semi-supervised node classification** on the Cora citation network: every node is a scientific paper described by a bag-of-words vector, an edge is a citation link, and the goal is to predict the paper's subject class when only 140 of the 2,708 papers are labelled.

Formally, let the graph be $G = (V, E, X)$ with $|V| = N$ nodes, adjacency matrix $A \in \{0,1\}^{N \times N}$ and feature matrix $X \in \mathbb{R}^{N \times F}$. Labels $y_i \in \{0,\ldots,C-1\}$ are known only for a small set V_L. The model must learn $f_\theta(A, X) \to P \in [0,1]^{N \times C}$ so that $\arg\max P_i$ matches y_i on nodes whose labels were never used during training.

Scope of the experiment. The report evaluates exactly the protocol that is implemented: the public Planetoid split of Cora (140 training, 500 validation, 1,000 test nodes, supplied with the data and not re-drawn), ten random seeds for weight initialisation and dropout, and checkpoint selection by validation loss. A single dataset is used, so the results do not show how the model behaves on other graphs, and no hyper-parameter search was performed.

1.1 Datasets used

Dataset	Role	Description
Toy graph (4 nodes)	Illustration and hand verification	3 edges, 2 features. Used with **fixed, hand-chosen weights** to show every intermediate matrix. No learning, no accuracy claimed.
Cora (2,708 nodes)	Trainable model and evaluation	5,278 undirected edges, 1,433 binary word features, 7 classes, public 140/500/1000 split.
60-node block-model graph	Supplementary (repository only)	Synthetic 3-community graph used for the gradient check and a small visual demonstration; see README.

2. Objectives

1. Represent a citation network as a graph with node-feature matrix, adjacency matrix and labels.
2. Derive the GCN propagation rule (self-loops, degree matrix, symmetric normalisation) and the equations of all three layers.
3. Implement every mathematical operation explicitly in NumPy, one function per formula, and verify intermediate matrices on a toy graph.
4. Derive back-propagation for the three layers by hand and verify it against numerical (finite-difference) gradients.
5. Train the three-layer GCN on Cora with the public split, select checkpoints by validation loss, and report held-out results over ten seeds with mean and sample standard deviation.
6. Quantify what the graph contributes by comparing against the identical network with message passing removed ($\hat{A} = I$).
7. Report accuracy, macro precision/recall/F1, confusion matrices and learning curves, and state the limits of the evidence.
8. Provide a PyTorch implementation of the same model and organise code, outputs and documentation in a repository.

3. Dataset and Graph Representation (Phase 1)

3.1 Graph fundamentals

A **graph** models entities and relations. **Nodes** are the entities, **edges** the relations. Unlike an image grid or a sentence, a graph has no canonical node ordering and nodes have different numbers of neighbours, so a convolution must be defined through the connectivity itself.

Object	Meaning	In Cora
Node set V	the N entities	2,708 papers
Edge set E	pairs of related nodes	5,278 citation links, treated as undirected
Adjacency matrix A	A_{ij} = 1 if i and j are connected; symmetric, zero diagonal	2,708 × 2,708, stored sparsely (avg. degree 3.90)
Feature matrix X	row i = feature vector x_i of node i	2,708 × 1,433 word indicators, row-normalised
Degree d_i	number of neighbours = row sum of A	no isolated nodes in Cora
Labels y	class of every node, known only on V_L	7 classes, 140 labelled training nodes

3.2 What the Cora graph means and how it is prepared

Each node is a machine-learning paper; its feature vector marks which of 1,433 dictionary words appear in it. An edge records that one paper cites another. Preparation steps (in `src/cora_data.py`): (i) the Planetoid files are read; (ii) edges are symmetrised so information can flow both ways along a citation and duplicates and self-references are removed (giving 5,278 unique edges); (iii) each feature row is divided by its sum (standard row normalisation); (iv) the fixed index sets for training, validation and test are taken from the files. Self-loops are *not* stored in A; they are added inside the normalisation (Section 6.1). The homophily of the graph – the fraction of edges joining two papers of the same class – is 0.810, which is the structural property that neighbourhood averaging exploits.

3.3 Fixed data partitions

Partition	Nodes	Share	Per-class counts (class 0 … 6)	Used for
Training	140	5.2%	[20, 20, 20, 20, 20, 20, 20]	loss and gradients
Validation	500	18.5%	[61, 36, 78, 158, 81, 57, 29]	checkpoint selection
Test	1000	36.9%	[130, 91, 144, 319, 149, 103, 64]	final reporting only
Total	2708	100%	[351, 217, 418, 818, 426, 298, 180]	

The partitions are the public split and are not re-drawn by the code. The training set is balanced (20 nodes per class); the test set is not, and the largest test class (class 3) has 319 of the 1,000 test nodes (31.9%), which is the accuracy of a constant prediction of that class. Because the test set has 1,000 nodes, one additional correct prediction changes accuracy by 0.1 percentage point.

3.4 Interpretation of the class labels

The data files provide the class as a one-hot column index 0–6. This report keeps the numeric names and does not attach subject names to them, because the mapping was not verified here.

4. Why a Graph Convolutional Network? (Phase 1)

A fully connected network classifies each paper from its own words only. In a citation network, however, papers on one topic cite each other, so the neighbours of a paper carry evidence about its topic. A GCN uses this evidence through **neighbourhood aggregation**: in each layer a node replaces its representation by a weighted average of its own and its neighbours' representations, followed by a learnable transformation.

Message passing in three steps. (1) Every neighbour j, and the node itself, sends its current feature vector. (2) The receiver forms a weighted sum, the weight being $1/\sqrt{(\tilde{d}_i \tilde{d}_j)}$. (3) The sum is multiplied by the shared weight matrix W, a bias is added and a non-linearity is applied. After one layer information has travelled one hop; after three layers a node's output depends on all nodes within three hops.

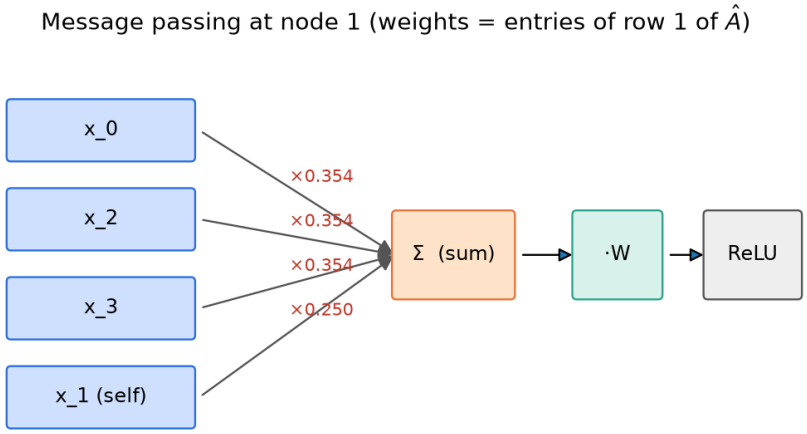


Figure 1 – Message passing at node 1 of the toy graph. The red numbers are the actual entries of row 1 of Â.

Ingredient	Role	Without it
Self-loops (A + I)	keep the node's own features in its update	own information lost; only neighbours contribute
Degree normalisation $\tilde{D}^{-1/2}(\cdot)\tilde{D}^{-1/2}$	stops high-degree nodes dominating; keeps scale stable across layers	sums grow with degree; unstable activations
Weight matrix $W^{(l)}$	learned linear map; changes width (1433 → 64 → 32 → 7)	nothing to learn; only fixed smoothing
ReLU	non-linearity between layers	three layers collapse to the single linear map $\hat{A}^3XW$
Bias $b^{(l)}$	per-feature offset	slightly less flexible

What this experiment does and does not show. The common claim that using the graph helps is tested here directly: the same three-layer network is trained with Â replaced by the identity matrix (so no information crosses edges), under the same split, seeds and hyper-parameters (Section 8.5). No claim is made about other graph models or about other datasets.

5. Methodology and Overall Architecture (Phase 1)

Workflow: load Cora → symmetrise, row-normalise features → compute Â once → three graph-convolution layers → softmax over 7 classes → cross-entropy on the 140 training nodes → pick the checkpoint with the lowest validation loss → report test metrics.

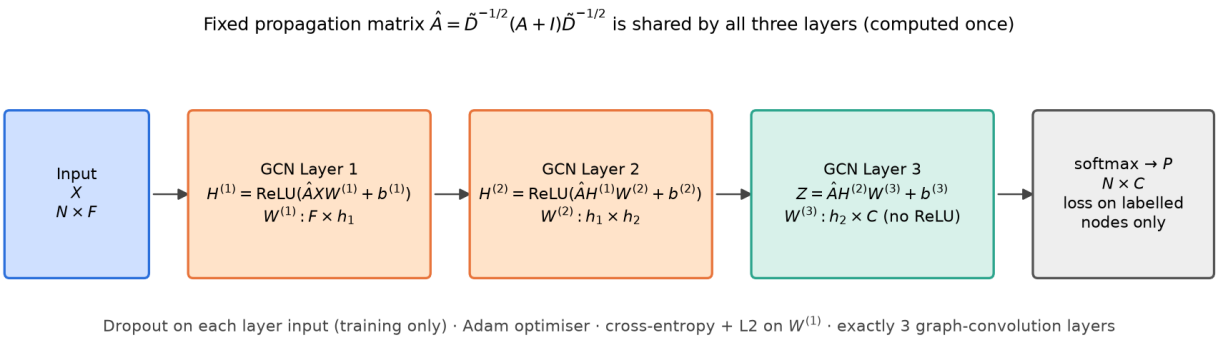


Figure 2 – Data flow of the three-layer GCN. h_1 = 64 and h_2 = 32 in the Cora experiment.

Table 1 – Tensor dimensions (N = 2,708 nodes, C = 7 classes).

Stage	Input	Output	Operation
GCN 1	N × 1433	N × 64	dropout on input, Â aggregation, linear, ReLU
GCN 2	N × 64	N × 32	dropout on input, Â aggregation, linear, ReLU
GCN 3	N × 32	N × 7	dropout on input, Â aggregation, linear (logits, no ReLU)
Softmax	N × 7	N × 7	row-wise class probabilities; loss only on the 140 labelled nodes

Table 2 – Trainable parameters, including biases.

Layer	Calculation	Parameters
GCN 1	1433 × 64 + 64	91,776
GCN 2	64 × 32 + 32	2,080
GCN 3	32 × 7 + 7	231
Total		94,087

The same pre-computed Â is used by all three layers. Layer 3 directly outputs class scores, so no extra dense layer is added and there are exactly three graph convolutions. A cost of depth is over-smoothing: repeated averaging makes neighbouring representations similar. The assignment fixes depth at three, so other depths were not studied.

6. Mathematical Model (Phase 2)

6.1 Adjacency, self-loops, degree matrix and symmetric normalisation

The graph is represented by $A \in \{0,1\}^{N\times N}$ ($A_{ij} = 1$ if i and j are connected) and $X \in \mathbb{R}^{N\times F}$. Self-loops are added so that every node is its own neighbour:

$$\tilde{A} = A + I_N \tag{1}$$

The degree matrix of the augmented graph is diagonal:

$$\tilde{D}_{ii} = \sum_j \tilde{A}_{ij} = 1 + d_i \tag{2}$$

The symmetric normalisation divides each connection by the geometric mean of the two endpoint degrees:

$$\hat{A} = \tilde{D}^{-1/2}\,\tilde{A}\,\tilde{D}^{-1/2}, \qquad \hat{A}_{ij} = \frac{\tilde{A}_{ij}}{\sqrt{\tilde{d}_i\,\tilde{d}_j}} \tag{3}$$

Toy example (edges 0–1, 1–2, 1–3): the degrees with self-loops are 2, 4, 2, 2, so $\hat{A}_{10} = 1/\sqrt{(4\cdot 2)} = 0.3536$ and $\hat{A}_{11} = 1/4 = 0.25$.

A

0	1	0	
1	0	1	
0	1	0	
0	1	0	

Ã = A + I

1	1	0	
1	1	1	
0	1	1	
0	1	1	

D̃

2	0	0	0
0	1	4	0
0	0	0	2
0	1	0	0

Â

0.50000	0.3536	0	0
0.35360	0.2500	0.3536	0.3536
0	0	0.3536	0.5000	0
0	2	0.3536	0	0.5000

6.2 Derivation of the graph-convolution equation

Step 1 – spectral convolution. With the normalised Laplacian $L = I - D^{-1/2}AD^{-1/2} = U\Lambda U^T$, filtering a node signal x with g_θ is

$$g_\theta \star x = U\,g_\theta(\Lambda)\,U^T x \tag{4}$$

This needs an eigen-decomposition and is not local. **Step 2 – Chebyshev polynomials** of $\tilde{\Lambda} = 2\Lambda/\lambda_{max} - I$ approximate the filter by a K-hop local polynomial of L. **Step 3 – first order** ($K = 1$, $\lambda_{max} \approx 2$):

$$g_{\theta'} \star x \approx \theta'_0 x - \theta'_1 D^{-1/2}AD^{-1/2}x \tag{5}$$

Step 4 – tie the two parameters ($\theta = \theta'_0 = -\theta'_1$):

$$g_\theta \star x \approx \theta(I + D^{-1/2}AD^{-1/2})x \tag{6}$$

Step 5 – renormalisation trick. The operator in (6) has eigenvalues in [0, 2]; stacking layers can make values explode or vanish. Replacing it by $\hat{A}$ of (3) (eigenvalues in (−1, 1]) and generalising from one scalar filter to F input and F' output channels (θ becomes $W \in \mathbb{R}^{F\times F'}$) gives the **GCN layer**:

$$H^{(l+1)} = \sigma\!\left(\hat{A}\,H^{(l)}\,W^{(l)} + \mathbf{1}\,b^{(l)T}\right), \qquad H^{(0)} = X \tag{7}$$

Row i of (7) is the node-level (message-passing) form:

$$h_i^{(l+1)} = \sigma\!\left(\sum_{j\,\in\,\mathcal{N}(i)\,\cup\,\{i\}} \frac{1}{\sqrt{\tilde{d}_i\tilde{d}_j}}\,h_j^{(l)}W^{(l)} + b^{(l)}\right) \tag{8}$$

6.3 Activation and dropout

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = 1 \text{ if } z > 0, \quad 0 \text{ otherwise} \tag{9}$$

ReLU is applied element-wise after aggregation and transformation in layers 1 and 2. Inverted dropout (rate p = 0.5) is applied to the input of each layer during training only:

$$\mathrm{Drop}_p(U) = \frac{M \odot U}{1-p}, \qquad M_{ij} \sim \mathrm{Bernoulli}(1-p) \tag{10}$$

The mask acts on feature entries, not on nodes or edges, and is the identity at evaluation time.

6.4 Equations of the three layers

$$H^{(1)} = \mathrm{ReLU}\left(\hat{A}\, X\, W^{(1)} + b^{(1)}\right), \quad W^{(1)} \in \mathbb{R}^{1433 \times 64} \tag{11}$$

$$H^{(2)} = \mathrm{ReLU}\left(\hat{A}\, H^{(1)}\, W^{(2)} + b^{(2)}\right), \quad W^{(2)} \in \mathbb{R}^{64 \times 32} \tag{12}$$

$$Z = \hat{A}\, H^{(2)}\, W^{(3)} + b^{(3)}, \quad W^{(3)} \in \mathbb{R}^{32 \times 7} \tag{13}$$

In one line: $Z = \hat{A} \cdot \mathrm{ReLU}(\hat{A} \cdot \mathrm{ReLU}(\hat{A}XW^{(1)})\, W^{(2)}) \cdot W^{(3)}$. Without the two ReLUs this equals $\hat{A}^3XW^{(1)}W^{(2)}W^{(3)}$, a linear model.

6.5 Output layer, loss function and parameter learning

$$P_{ic} = \frac{\exp(Z_{ic})}{\sum_{c'} \exp(Z_{ic'})} \tag{14}$$

Training is semi-supervised: the cross-entropy uses only the labelled nodes V_L, but $\hat{A}$ lets their gradient reach every node within three hops. Weight decay is applied to the first layer's weights.

$$\mathcal{L} = -\frac{1}{|V_L|} \sum_{i \in V_L} \log P_{i, y_i} + \frac{\lambda}{2} \left\| W^{(1)} \right\|_F^2 \tag{15}$$

Back-propagation. Let M_L be the 0/1 training-node mask and Y the one-hot labels. The gradient of softmax followed by cross-entropy is simple, and $\hat{A}$ is symmetric, so $\hat{A}^T = \hat{A}$:

$$\delta^{(3)} = \frac{\partial \mathcal{L}}{\partial Z} = \frac{M_L \odot (P - Y)}{|V_L|} \tag{16}$$

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \left(\hat{A} H^{(l-1)}\right)^T \delta^{(l)}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \mathbf{1}^T \delta^{(l)} \tag{17}$$

$$\delta^{(l-1)} = \left[\hat{A}\, \delta^{(l)}\, W^{(l)T}\right] \odot \mathrm{mask}^{(l)} \odot \mathbb{1}[Z^{(l-1)} > 0] \tag{18}$$

where $\mathrm{mask}^{(l)}$ is the dropout mask of layer l. The L2 term adds $\lambda W^{(1)}$ to $\partial L/\partial W^{(1)}$. Parameters are updated with **Adam** (implemented from scratch): $m \leftarrow \beta_1 m + (1-\beta_1) g$, $v \leftarrow \beta_2 v + (1-\beta_2) g^2$, $\theta \leftarrow \theta - \eta \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$, with bias-corrected $\hat{m}$, $\hat{v}$. Equations (16)–(18) are checked against finite differences in Section 8.1.

6.6 Evaluation metrics

Let C_{ab} be the number of test nodes of actual class a predicted as b. Accuracy = $\Sigma_a C_{aa} / \Sigma_{a,b} C_{ab}$. For each class c, precision = $TP_c/(TP_c+FP_c)$, recall = $TP_c/(TP_c+FN_c)$, $F1_c = 2PR/(P+R)$; undefined ratios are set to 0. **Macro** averages give all 7 classes equal weight; macro F1 is the mean of the seven class F1 scores. Over R seeds the mean is $m = (1/R)\Sigma m_r$ and the *sample* standard deviation is $s = \sqrt{(\Sigma(m_r - m)^2 / (R - 1))}$ with R = 10. This spread measures variation due to initialisation and dropout on one fixed split, not uncertainty about the wider population of papers.

6.7 Algorithm and pseudocode

```
Algorithm: three-layer vanilla GCN, semi-supervised node classification
Input : A (N x N, sparse), X (N x F), labels y on training set V_L, widths h1, h2, epochs T
Output: parameters of the epoch with the lowest validation loss; class probabilities P

 1  A_tilde <- A + I                       # self-loops
 2  d_i     <- sum_j A_tilde[i, j]         # degrees
 3  A_hat   <- D^{-1/2} A_tilde D^{-1/2}   # symmetric normalisation, computed once
 4  Initialise W_l with Xavier-uniform, b_l <- 0               (l = 1, 2, 3)
 5  for epoch = 1 .. T:
 6      Hin_1 <- X
 7      for l = 1 .. 3:                                       # FORWARD
 8          U_l <- dropout(Hin_l)
 9          Z_l <- A_hat @ U_l @ W_l + b_l
10          Hin_{l+1} <- ReLU(Z_l)  if l < 3  else  Z_l
11      P <- softmax(Z_3)
12      L <- cross_entropy(P[V_L], y[V_L]) + (lambda/2) ||W_1||^2
13      delta <- (P - Y) * mask_L / |V_L|                     # BACKWARD
14      for l = 3, 2, 1:
15          gW_l <- (A_hat @ U_l)^T @ delta ;   gb_l <- column-sum(delta)
16          if l > 1: delta <- (A_hat @ delta @ W_l^T) * dropout_mask_l * [Z_{l-1} > 0]
17      Adam update of all W_l, b_l
18      evaluate (no dropout): validation loss; save parameters if it is the lowest so far
19  restore the saved parameters; return softmax(Z_3) for every node
```

6.8 Complexity

With sparse Â holding $|E|' = 2|E| + N$ non-zero entries, one layer costs $O(|E|' \cdot F_{in} + N \cdot F_{in} \cdot F_{out})$ operations: the first term is the aggregation, the second the dense transform. Cora's Â has 13264 non-zeros instead of $2{,}708^2 \approx 7.3$ million entries, which is why sparse storage is used.

6.9 Numerical worked example (fixed weights – no training)

The 4-node toy graph has features X below. The weights are **hand-chosen constants** with zero biases. This demonstrates only the mechanics of the forward pass; **the weights are not trained and the resulting probabilities carry no accuracy information**.

X

1	0
0	1
1	1
2	0

$W^{(1)}$ 2×3

1	-1	0.5000
0.5000	1	-0.5000

$W^{(2)}$ 3×2

1	-1
1	1
-2	1

$W^{(3)}$ 2×2

1	-1
-1	1

Hand calculation, node 1, layer 1. Row 1 of Â is [1/√8, 1/4, 1/√8, 1/√8] = [0.3536, 0.25, 0.3536, 0.3536].

$(\hat{A}X)_1$ = 0.3536·[1,0] + 0.25·[0,1] + 0.3536·[1,1] + 0.3536·[2,0] = [1.4142, 0.6036], matching the code. Multiplying by $W^{(1)}$ gives [1.7160, -0.8107, 0.4053]; ReLU turns the negative entry to 0, so $H^{(1)}_1$ = [1.7160, 0.0000, 0.4053].

ÂX

0.5000	0.3536
1.4142	0.6036
0.5000	0.8536
1	0.3536

ÂX$W^{(1)}$

0.6768	-0.1464	0.0732
1.7160	-0.8107	0.4053
0.9268	0.3536	-0.1768
1.1768	-0.6464	0.3232

$H^{(1)}$

0.6768	0	0.0732
1.7160	0	0.4053
0.9268	0.3536	0
1.1768	0	0.3232

$\hat{A}H^{(1)}$

0.9451	0	0.1799
1.4120	0.1250	0.2415
1.0701	0.1768	0.1433
1.1951	0	0.3049

$\hat{A}H^{(1)}W^{(2)}$

0.5852	-0.7652
1.0540	-1.0455
0.9602	-0.7500
0.5852	-0.8902

$H^{(2)}$

0.5852	0
1.0540	0
0.9602	0
0.5852	0

$\hat{A}H^{(2)}$

0.6653	0
1.0168	0
0.8528	0
0.6653	0

Z

0.6653	-0.6653
1.0168	-1.0168
0.8528	-0.8528
0.6653	-0.6653

$P = \text{softmax}(Z)$

0.7909	0.2091
0.8843	0.1157
0.8463	0.1537
0.7909	0.2091

Checks visible in the output: ReLU clips negative entries in layers 1 and 2; every row of P sums to 1; the hub node 1 receives the largest aggregated values because it has three neighbours.

7. Training Protocol and Reproducibility

Table 3 – Settings used in the recorded experiment.

Setting	Value
Dataset	Cora, public Planetoid split: 140 train / 500 validation / 1,000 test
Layer widths	1433 → 64 → 32 → 7 (three graph convolutions)
Initialisation	Xavier/Glorot uniform for weights, zeros for biases
Dropout	0.5 on the input of every layer (training only)
Optimiser	Adam (own implementation), learning rate 0.01, β = (0.9, 0.999), ε = 1e-8
Weight decay	0.0005 on $W^{(1)}$ (added to the gradient)
Epochs	200 per seed, full batch; training is never stopped early
Checkpoint rule	lowest validation cross-entropy (equation 19)
Seeds	42, 123, 456, 7, 8, 9, 10, 11, 12, 13
Baseline	identical network and settings with $\hat{A} = I$ (no edges used)
Hardware / libraries	CPU; Python, NumPy, SciPy (no deep-learning framework needed for the reported results)

The selected epoch for run r is

$$t_r^* = \arg\min_{t \in \{1, \ldots, 200\}} \mathcal{L}_{\text{val}}(\theta_{r,t}), \tag{19}$$

where L_{val} is the cross-entropy over the 500 validation nodes with dropout off. The test set supplies no gradient and plays no role in this choice. The hyper-parameters (widths, learning rate, dropout, weight decay) were fixed before the final runs and were not tuned on the test set; they were not searched systematically either, so a different setting could give different numbers. Seeds control weight initialisation and dropout masks; the data split is the same for every run.

Reproducibility: `python src/run_cora.py` regenerates every number in Section 8. Run time was roughly 40–70 s per seed on one CPU core.

8. Experimental Results (Phase 3)

8.1 Verification of the hand-derived gradients

To confirm that equations (16)–(18) are implemented correctly, `tests/gradient_check.py` compares them with central finite differences (step 10^{-5}) for 30 randomly selected entries across all six parameter tensors (dropout off, small graph). Output stored in `outputs/gradient_check_output.txt`:

```
Worst relative error over 30 checked entries: 2.31e-08
GRADIENT CHECK PASSED
```

8.2 Program output

The listing is the unmodified output of `src/run_cora.py` (file `outputs/cora_output.txt`).

```
Cora: nodes=2708 undirected edges=5278 features=1433 classes=7
split train/val/test = 140/500/1000  | trainable parameters = 94,087
class counts: [351, 217, 418, 818, 426, 298, 180] | test class counts: [130, 91, 144, 319, 149, 103, 64]
homophily = 0.810   mean degree = 3.90
config: {'hidden': (64, 32), 'epochs': 200, 'lr': 0.01, 'dropout': 0.5, 'wd': 0.0005, 'select': 'val_loss'}
--------------------------------------------------------------------------------
[GCN] seed  42 | best epoch  99 | val loss 0.6838 | test loss 0.5985 | test acc 0.8160 | macro-F1 0.8008 | 38s
[MLP] seed  42 | best epoch  50 | val loss 1.3254 | test loss 1.3354 | test acc 0.5420 | macro-F1 0.5256 | 32s
[GCN] seed 123 | best epoch  81 | val loss 0.7181 | test loss 0.6116 | test acc 0.8290 | macro-F1 0.8117 | 40s
[MLP] seed 123 | best epoch  47 | val loss 1.3590 | test loss 1.3336 | test acc 0.5310 | macro-F1 0.5156 | 32s
[GCN] seed 456 | best epoch  56 | val loss 0.6558 | test loss 0.5665 | test acc 0.8220 | macro-F1 0.8091 | 37s
[MLP] seed 456 | best epoch  39 | val loss 1.5034 | test loss 1.4530 | test acc 0.5180 | macro-F1 0.5027 | 33s
[GCN] seed   7 | best epoch  58 | val loss 0.6716 | test loss 0.6126 | test acc 0.8130 | macro-F1 0.8029 | 38s
[MLP] seed   7 | best epoch  94 | val loss 1.2953 | test loss 1.3356 | test acc 0.5680 | macro-F1 0.5543 | 32s
[GCN] seed   8 | best epoch  79 | val loss 0.6583 | test loss 0.5912 | test acc 0.8250 | macro-F1 0.8133 | 36s
[MLP] seed   8 | best epoch  44 | val loss 1.3100 | test loss 1.3079 | test acc 0.5730 | macro-F1 0.5556 | 31s
[GCN] seed   9 | best epoch 137 | val loss 0.7140 | test loss 0.5992 | test acc 0.8200 | macro-F1 0.8082 | 37s
[MLP] seed   9 | best epoch  55 | val loss 1.3711 | test loss 1.3685 | test acc 0.5310 | macro-F1 0.5155 | 30s
[GCN] seed  10 | best epoch  58 | val loss 0.6805 | test loss 0.5902 | test acc 0.8140 | macro-F1 0.8082 | 38s
[MLP] seed  10 | best epoch  66 | val loss 1.3976 | test loss 1.3583 | test acc 0.5350 | macro-F1 0.5219 | 31s
[GCN] seed  11 | best epoch  98 | val loss 0.6718 | test loss 0.6154 | test acc 0.8030 | macro-F1 0.7913 | 37s
[MLP] seed  11 | best epoch  45 | val loss 1.3639 | test loss 1.3453 | test acc 0.5320 | macro-F1 0.5155 | 29s
[GCN] seed  12 | best epoch  95 | val loss 0.6561 | test loss 0.6171 | test acc 0.8060 | macro-F1 0.7918 | 38s
[MLP] seed  12 | best epoch  78 | val loss 1.3151 | test loss 1.3510 | test acc 0.5610 | macro-F1 0.5489 | 32s
[GCN] seed  13 | best epoch 127 | val loss 0.6703 | test loss 0.6299 | test acc 0.8120 | macro-F1 0.8014 | 37s
[MLP] seed  13 | best epoch  49 | val loss 1.2944 | test loss 1.2642 | test acc 0.5680 | macro-F1 0.5475 | 31s
--------------------------------------------------------------------------------
MLP acc: 0.5459 Â± 0.0197 | macro_precision: 0.5264 Â± 0.0202 | macro_recall: 0.5538 Â± 0.0140 | macro_f1: 0.5303 Â± 0.0193 | test_loss: 1.3453 Â± 0.0479 | best_va
GCN acc: 0.8160 Â± 0.0082 | macro_precision: 0.7991 Â± 0.0071 | macro_recall: 0.8129 Â± 0.0109 | macro_f1: 0.8039 Â± 0.0077 | test_loss: 0.6032 Â± 0.0180 | best_va

Seed 42 GCN confusion matrix (rows actual, cols predicted):
 [[ 93   5   2  11   8   8   3]
 [  3  74   5   6   0   3   0]
 [  1   5 130   8   0   0   0]
 [ 10   6   6 269  19   6   3]
 [ 13   2   0  11 118   4   1]
 [  4   2   4   2   0  86   5]
 [ 10   0   0   1   0   7  46]]
GCN per-class F1 (seed 42): [0.705, 0.8, 0.893, 0.858, 0.803, 0.793, 0.754]
```

8.3 Checkpoint selection

Table 4 – Selected checkpoints of the GCN (lowest validation loss).

Seed	Selected epoch	Best validation loss	Validation accuracy at that epoch	Train accuracy
42	99	0.6838	0.7980	1.0000
123	81	0.7181	0.7840	1.0000
456	56	0.6558	0.8020	0.9929
7	58	0.6716	0.8020	1.0000
8	79	0.6583	0.7980	1.0000
9	137	0.7140	0.7840	1.0000
10	58	0.6805	0.7960	0.9929
11	98	0.6718	0.7960	1.0000
12	95	0.6561	0.7960	1.0000
13	127	0.6703	0.8020	1.0000

The selected epochs range from 56 to 137 out of 200, so the rule is active: the last epoch is not automatically the best, and validation loss rises again later in training as the model fits the 140 training nodes (training accuracy is at or near 1.0).

8.4 Learning curves

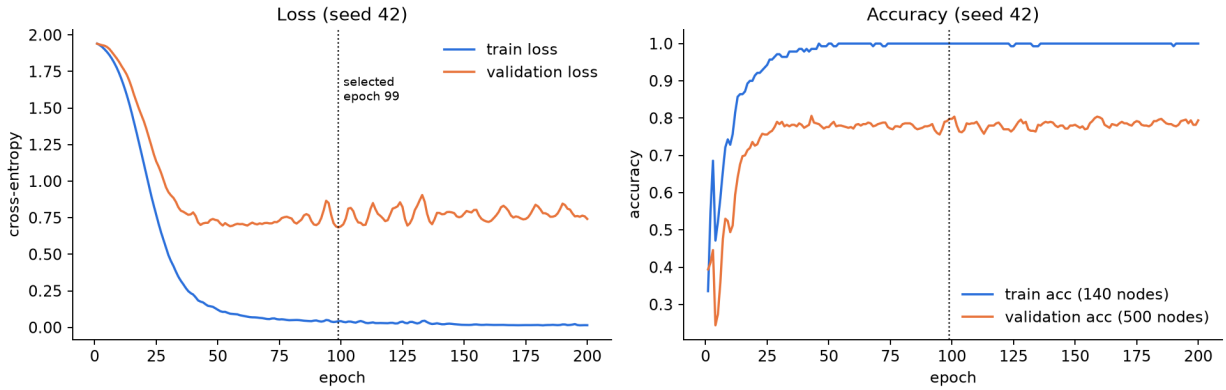


Figure 3 – Training and validation loss/accuracy for seed 42 (full 200 epochs). The dotted line marks the selected checkpoint.

8.5 Individual test runs, mean and sample standard deviation

Table 5 – Test metrics of the selected GCN checkpoints (1,000 test nodes). Macro metrics average over the 7 classes.

Seed	Test loss	Accuracy	Macro precision	Macro recall	Macro F1
42	0.5985	0.8160	0.8000	0.8029	0.8008
123	0.6116	0.8290	0.8071	0.8240	0.8117
456	0.5665	0.8220	0.8010	0.8223	0.8091
7	0.6126	0.8130	0.7951	0.8156	0.8029
8	0.5912	0.8250	0.8122	0.8219	0.8133
9	0.5992	0.8200	0.8036	0.8133	0.8082
10	0.5902	0.8140	0.7974	0.8230	0.8082
11	0.6154	0.8030	0.7899	0.7958	0.7913
12	0.6171	0.8060	0.7931	0.7963	0.7918
13	0.6299	0.8120	0.7915	0.8143	0.8014

Table 6 – Mean ± sample standard deviation over 10 seeds, GCN versus the same network without the graph (Â = I).

Metric	No graph (Â = I)	Three-layer GCN
Best validation loss	1.3535 ± 0.0632	0.6780 ± 0.0222
Test loss	1.3453 ± 0.0479	0.6032 ± 0.0180
Accuracy	0.5459 ± 0.0197	0.8160 ± 0.0082
Macro precision	0.5264 ± 0.0202	0.7991 ± 0.0071
Macro recall	0.5538 ± 0.0140	0.8129 ± 0.0109
Macro F1	0.5303 ± 0.0193	0.8039 ± 0.0077

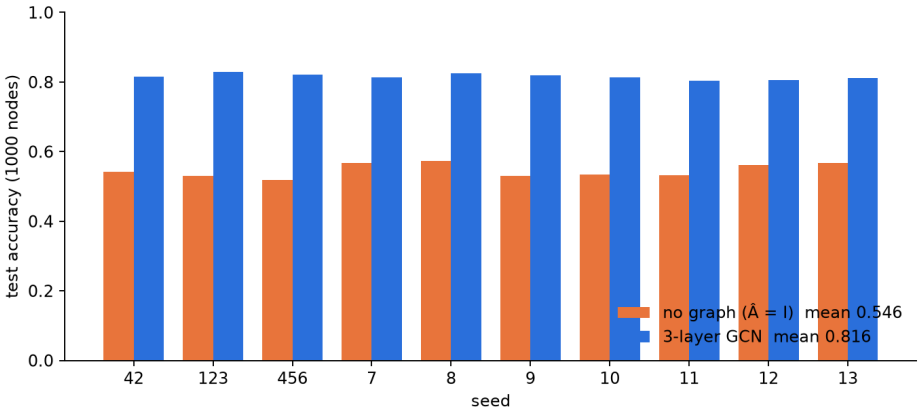


Figure 4 – Test accuracy per seed, with and without message passing.

Averaged over ten seeds the GCN reaches 81.6% test accuracy (s = 0.8 points) against 54.6% (s = 2.0 points) for the identical network without edges, a difference of 27.0 points; the GCN is ahead for each of the ten seeds (smallest per-seed margin 24.4 points). The comparison uses the same split, features, widths and training settings, so the gap is attributable to neighbourhood aggregation under these settings. It is a statement about this split of Cora only; the standard deviations describe variation across seeds, not across datasets or splits.

8.6 Confusion matrices and class-specific behaviour

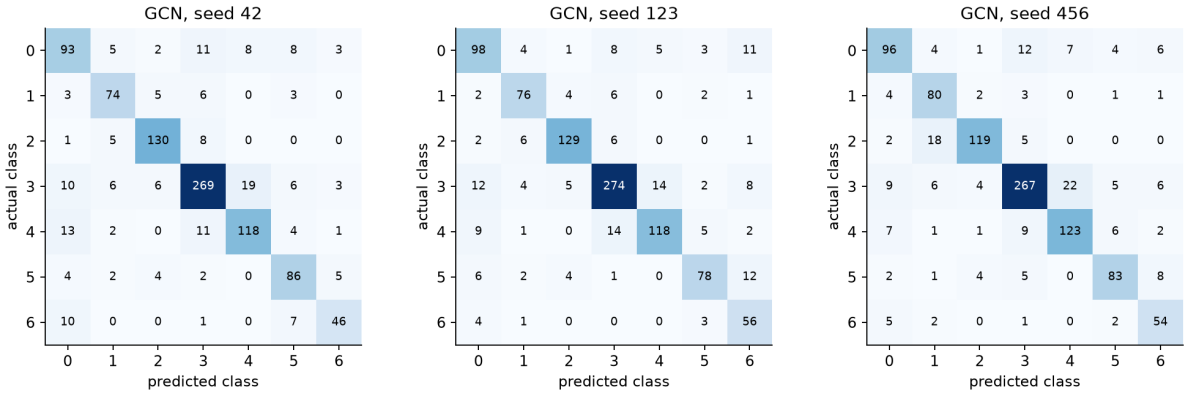


Figure 5 – Test confusion matrices of the GCN for the first three seeds. Rows: actual class; columns: predicted class.

For seed 42 the per-class F1 scores are [0.705, 0.8, 0.893, 0.858, 0.803, 0.793, 0.754]; per-class recall is [0.715, 0.813, 0.903, 0.843, 0.792, 0.835, 0.719]. The best-recognised class is 2 and the weakest is 0. The largest single confusion is 3 actual → 4 predicted (19 nodes). A single accuracy value therefore hides substantial differences between classes.

8.7 What the network learns: embeddings

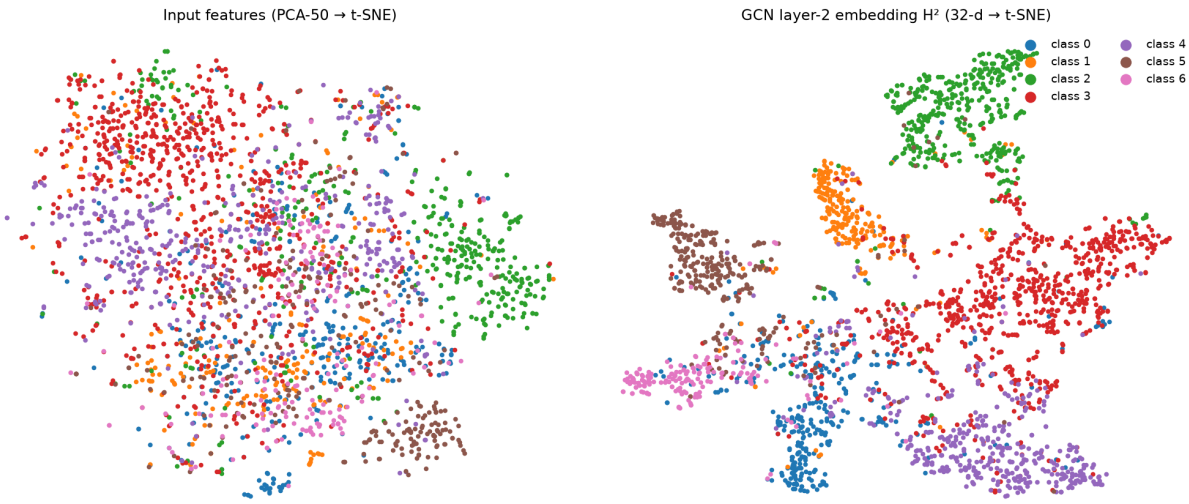


Figure 6 – t-SNE of all 2,708 nodes, coloured by true class: raw input features (left) and the 32-dimensional layer-2 representation of the seed-42 GCN (right). t-SNE is a visual aid; distances in it are not quantitative.

The right panel shows more compact class groups than the left. This is an illustration, not a measurement: t-SNE depends on its settings and no cluster-quality score was computed.

9. Model Code and Formula-to-Code Map

The core of the trainable model (`src/gcn_numpy.py`, excerpt). Each marked line implements one equation.

```
def forward(self, A_hat, X, train=False):
    H = X
    for l in range(3):
        if train: m = (rng.random(H.shape) > p) / (1 - p); H = H * m   # Eq. 10  dropout
        AH = A_hat @ H                       # neighbourhood aggregation    (Eq. 7)
        Z  = AH @ self.W[l] + self.b[l]      # linear transform + bias
        H  = relu(Z) if l < 2 else Z         # Eq. 9 (layers 1-2); logits in layer 3
    return softmax(H)                        # Eq. 14

def backward(self, cache, y, train_mask):
    dZ = (P - Y) * train_mask[:, None] / n_tr  # Eq. 16
    for l in (2, 1, 0):
        gW[l] = cache["AH"][l].T @ dZ          # Eq. 17  (A_hat H)^T delta
        gb[l] = dZ.sum(axis=0, keepdims=True)
        if l > 0:
            dH = A.T @ (dZ @ self.W[l].T) * dropout_mask   # Eq. 18  A_hat symmetric
            dZ = dH * (cache["Z"][l - 1] > 0)              # ReLU derivative
    gW[0] += wd * self.W[0]                  # gradient of the L2 term in Eq. 15
```

Formula	Code	Note
$\tilde{A} = A + I$ (1)	gcn_ops.add_self_loops / sparse identity	adds the diagonal
$\tilde{D}_{ii} = \Sigma_j \tilde{A}_{ij}$ (2)	gcn_ops.degree_matrix, row sums	
$\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ (3)	gcn_ops.sym_normalize	sparse diag($d^{-1/2}$) multiplied on both sides
$H' = \sigma(\hat{A}HW + b)$ (7)	gcn_ops.gcn_layer; GCN3.forward	`A_hat @ H` aggregates; `@ W` transforms
ReLU (9), softmax (14)	gcn_ops.relu, softmax	softmax subtracts row max for stability
Loss (15)	gcn_ops.cross_entropy, GCN3.loss	masked mean over labelled nodes + L2
Gradients (16)–(18)	GCN3.backward	checked by tests/gradient_check.py
Adam	GCN3.step	from scratch with bias correction
Checkpoint rule (19)	train_gcn(select='val_loss')	parameters of lowest validation loss restored
Metrics (Sec. 6.6)	gcn_numpy.confusion_and_macro	undefined ratios set to 0

Fixed-weight demonstration versus trainable GCN

	Fixed-weight demo (Section 6.9)	Trainable GCN (Section 8)
File	src/demo_fixed_weights.py	src/gcn_numpy.py, src/run_cora.py
Graph	4 nodes, 2 features	Cora: 2,708 nodes, 1,433 features, 7 classes
Weights	hand-picked constants, never updated	Xavier init, Adam updates
Meaningful predictions?	No – checks arithmetic only	Yes – evaluated on 1,000 held-out nodes

PyTorch version. `src/gcn_torch.py` defines the same model with autograd (`GraphConv` layers computing `A_hat @ H @ W + b`); `tests/test_torch_matches_numpy.py` checks that its forward pass and all gradients agree with the NumPy code. Neither file has been executed by the author of this package (no PyTorch installation was possible in the preparation environment); please run the test before using the PyTorch file.

Conclusion

A three-layer vanilla GCN was derived from the spectral graph filter, implemented with one function per equation, with hand-derived back-propagation verified against numerical gradients, and evaluated on the public Cora split. Over ten seeds it obtained 81.6% ± 0.8 test accuracy and 0.804 ± 0.008 macro F1, compared with 54.6% ± 2.0 and 0.530 ± 0.019 for the same network without edges, so neighbourhood aggregation made a clear difference on this split.

Limits of the evidence. One dataset and one fixed split were used; the seeds vary only initialisation and dropout. Hyper-parameters were not systematically tuned. No comparison with other graph architectures or with deeper or shallower GCNs was made, so nothing is claimed about them. Cora is strongly homophilous, which favours neighbourhood averaging; graphs where neighbours often have different labels may behave differently. The PyTorch implementation is untested here. The most informative next steps would be repeating the experiment with random splits or further datasets, and adding a depth study (which the three-layer constraint of this assignment excluded).

How to run

```
pip install -r requirements.txt
bash run_all.sh                          # everything below, in order
python src/demo_fixed_weights.py         # fixed-weight demo (Section 6.9)
python tests/gradient_check.py           # gradient verification (Section 8.1)
python src/run_cora.py                   # main experiment, ~12 min on one CPU core
python src/make_figures.py; python src/make_cora_figures.py; python src/build_report.py
python tests/test_torch_matches_numpy.py # needs PyTorch; not run by the package author
```

References

T. N. Kipf and M. Welling, *Semi-Supervised Classification with Graph Convolutional Networks*, ICLR 2017. Z. Yang, W. Cohen, R. Salakhutdinov, *Revisiting Semi-Supervised Learning with Graph Embeddings*, ICML 2016 (Planetoid split of Cora). M. Defferrard, X. Bresson, P. Vandergheynst, *Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering*, NeurIPS 2016. D. K. Hammond, P. Vandergheynst, R. Gribonval, *Wavelets on graphs via spectral graph theory*, 2011.